

Cài trên Pi

1.1. Pi4 model B + RPLidar A1M8

- Cài đặt Ubuntu Server 22 LTS (64-bit)

- + Hostname: robotanninh
- + Username/ Password: ubuntu/ abc123
- + Kết nối Lidar và Pi4b qua USB - USB micro B, cổng đen

- Cập nhật hệ điều hành:

```
sudo apt update && sudo apt upgrade -y  
sudo reboot
```

- Tạo file swap 4GB ram ảo:

```
sudo fallocate -l 4G /swapfile  
sudo chmod 600 /swapfile  
sudo mkswap /swapfile  
sudo swapon /swapfile  
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

- Thiết lập ngôn ngữ:

```
sudo apt install -y software-properties-common curl locales  
sudo locale-gen en_US en_US.UTF-8  
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8  
export LANG=en_US.UTF-8
```

- Kích hoạt kho Universe:

```
sudo add-apt-repository universe -y
```

1.2. Thêm ROS2 Humble repository:

```
sudo apt install -y curl  
export ROS_APT_SOURCE_VERSION=$(curl -s \  
https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \  
| grep -F '"tag_name"' | awk -F'"' '{print $4}')  
echo "ros-apt-source version: $ROS_APT_SOURCE_VERSION"  
  
curl -L -o /tmp/ros2-apt-source.deb \  
"https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VERSION}.deb" && echo $VERSION_CODENAME)_all.deb"  
sudo dpkg -i /tmp/ros2-apt-source.deb
```

- Cài ROS2 Humble:

```
sudo apt update  
sudo apt install -y ros-humble-ros-base ros-dev-tools
```

- Cài packages mở rộng (SLAM, Web, DDS):

```
sudo apt install -y \  
ros-humble-slam-toolbox \  
ros-humble-rosbridge-suite \  
ros-humble-rmw-cyclonedds-cpp
```

- Cấu hình môi trường:

```
echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc  
echo 'export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp' >> ~/.bashrc  
echo 'export ROS_DOMAIN_ID=42' >> ~/.bashrc  
source ~/.bashrc
```

- Cài bộ đồng bộ thời gian:

```
sudo apt update && sudo apt install chrony -y
```

1.3. Tạo Workspace & Tải mã nguồn của RPLidar A1M8::

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws/src  
git clone https://github.com/Slamtec/rplidar_ros2.git
```

1.4. Cấu hình lidar:

```
sudo usermod -aG dialout ubuntu  
sudo systemctl disable --now ModemManager
```

- Udev rule cố định /dev/rplidar: `sudo nano /etc/udev/rules.d/99-rplidar.rules`

```
KERNEL=="ttyUSB*", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", MODE="0666", GROUP="dialout", SYMLINK+="rplidar"
```

```
sudo udevadm control --reload-rules && sudo udevadm trigger
```

1.5 Cấu Hình I2C + MPU6050 GY-521:

- Kích hoạt I2C trong config.txt: `sudo nano /boot/firmware/config.txt`

```
dtoverlay=i2c_arm_baudrate=400000
```

- Nạp module i2c-dev khi boot:

```
echo 'i2c-dev' | sudo tee /etc/modules-load.d/i2c.conf
```

- Cài công cụ & thư viện:

```
sudo apt install -y i2c-tools python3-pip  
pip3 install smbus2
```

- Phân quyền I2C cho user ubuntu:

```
sudo groupadd -f i2c  
sudo usermod -aG i2c ubuntu  
echo 'KERNEL=="i2c-[0-9]*", GROUP="i2c", MODE="0660" \  
| sudo tee /etc/udev/rules.d/99-i2c.rules  
sudo udevadm control --reload-rules && sudo udevadm trigger
```

1.6. Tạo Package web_pose

```
cd ~/ros2_ws/src  
ros2 pkg create web_pose \  
--build-type ament_python \  
--dependencies rcpp geometry_msgs tf2_ros tf2_geometry_msgs nav_msgs sensor_msgs \  
--license Apache-2.0  
  
mkdir -p ~/ros2_ws/src/web_pose/launch  
mkdir -p ~/ros2_ws/config
```

1.8. Các chức năng về scan

```
sudo apt update  
sudo apt install ros-humble-laser-filters -y
```

- Tạo cấu hình scan: nano ~/ros2_ws/config/laser_filter.yaml

```
scan_to_scan_filter_chain:
  ros__parameters:
    filter1:
      name: range_filter
      type: laser_filters/LaserScanRangeFilter
      params:
        use_message_range_limits: false
        lower_threshold: 0.15
        upper_threshold: 12.0
        lower_replacement_value: .inf
        upper_replacement_value: .inf
```

1.9. Tạo file đọc IMU: nano ~/ros2_ws/src/web_pose/web_pose/mpu6050_driver.py

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Imu
import smbus2
import math
import time

MPU6050_ADDR = 0x68
PWR_MGMT_1 = 0x6B
GYRO_CONFIG = 0x1B
GYRO_XOUT_H = 0x43

GYRO_SCALE = 131.0
DEG2RAD = math.pi / 180.0
SCALE_FACTOR = DEG2RAD / GYRO_SCALE
CALIBRATE_N = 150

class MPU6050Driver(Node):
    def __init__(self):
        super().__init__('mpu6050_driver')
        self.bus = smbus2.SMBus(1)
        self.bus.write_byte_data(MPU6050_ADDR, PWR_MGMT_1, 0x00)
        time.sleep(0.1)
        self.bus.write_byte_data(MPU6050_ADDR, GYRO_CONFIG, 0x00)
        time.sleep(0.05)
        self._gz_offset = self._calibrate()
        self.pub = self.create_publisher(Imu, 'imu/data', 10)
        self.create_timer(0.02, self._publish)
        self.get_logger().info(f'MPU6050 ready: offset={math.degrees(self._gz_offset):.3f} deg/s')

    def _read_i16(self, reg: int) -> int:
        d = self.bus.read_i2c_block_data(MPU6050_ADDR, reg, 2)
        v = (d[0] << 8) | d[1]
        return v - 65536 if v >= 0x8000 else v

    def _calibrate(self) -> float:
        total = 0.0
        for _ in range(CALIBRATE_N):
            total += self._read_i16(GYRO_XOUT_H + 4)
            time.sleep(0.02)
        return (total / CALIBRATE_N) * SCALE_FACTOR

    def _publish(self):
        try:
            d = self.bus.read_i2c_block_data(MPU6050_ADDR, GYRO_XOUT_H, 6)
            v = (d[4] << 8) | d[5]
            gz_raw = v - 65536 if v >= 0x8000 else v
            gz_rads = (gz_raw * SCALE_FACTOR) - self._gz_offset

            msg = Imu()
            msg.header.stamp = self.get_clock().now().to_msg()
            msg.header.frame_id = 'imu_link'
            msg.angular_velocity.z = float(gz_rads)
            msg.angular_velocity.covariance = [
                1e6, 0.0, 0.0,
                0.0, 1e6, 0.0,
                0.0, 0.0, 1e-4
            ]
            msg.linear_acceleration_covariance[0] = -1.0
            msg.orientation_covariance[0] = -1.0
            self.pub.publish(msg)
        except Exception as e:
            self.get_logger().warn(f'IMU Error: {e}', throttle_duration_sec=3.0)

def main():
```

```

rclpy.init()
node = MPU6050Driver()
try:
    rclpy.spin(node)
except KeyboardInterrupt:
    pass
finally:
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

1.10. Tạo file giao tiếp STM32: nano ~/ros2_ws/src/web_pose/web_pose/uart_bridge.py

```

#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
from geometry_msgs.msg import Twist
from std_msgs.msg import String
import serial
import time

LIN_THRESH = 0.05
ANG_THRESH = 0.15
SERIAL_PORT = '/dev/ttyAMA0'
BAUD = 115200

def twist_to_cmd(lx: float, az: float) -> str:
    fwd = lx > LIN_THRESH
    back = lx < -LIN_THRESH
    lft = az > ANG_THRESH
    rgt = az < -ANG_THRESH

    if fwd and lft: return 'UP_LEFT'
    if fwd and rgt: return 'UP_RIGHT'
    if fwd: return 'ON'
    if back and lft: return 'DOWN_LEFT'
    if back and rgt: return 'DOWN_RIGHT'
    if back: return 'BACK'
    if lft: return 'LEFT'
    if rgt: return 'RIGHT'
    return 'OFF'

class UartBridge(Node):
    def __init__(self):
        super().__init__('uart_bridge')
        self._last_cmd = ""
        self._auto_mode = False
        self.ser = None
        self._open_serial(initial=True)
        self._reconnect_timer = self.create_timer(2.0, self._check_serial)

        qos = QoSProfile(
            reliability=ReliabilityPolicy.RELIABLE,
            history=HistoryPolicy.KEEP_LAST,
            depth=1
        )
        self.create_subscription(Twist, '/cmd_vel', self._on_vel, qos)
        self.create_subscription(String, '/robot_mode', self._on_mode, 10)

    def _open_serial(self, initial: bool = False) -> bool:
        try:
            self.ser = serial.Serial(SERIAL_PORT, BAUD, timeout=1)
            time.sleep(2.0 if initial else 0.2)
            self._last_cmd = ""
            return True
        except Exception as e:
            self.ser = None
            self.get_logger().warn(f'Serial Error: {e}')
            return False

    def _check_serial(self):
        if self.ser is None or not self.ser.is_open:
            self._open_serial(initial=False)

    def _write(self, cmd: str):
        if cmd == self._last_cmd:
            return
        if self.ser is None or not self.ser.is_open:

```

```

        return
    try:
        self.ser.write(f'{cmd}\n'.encode('utf-8'))
        self.ser.flush()
        self._last_cmd = cmd
    except Exception as e:
        self.get_logger().error(f'Write error: {e}')
    try:
        self.ser.close()
    except Exception:
        pass
    self.ser = None

def _on_vel(self, msg: Twist):
    if self._auto_mode:
        return
    self._write(twist_to_cmd(msg.linear.x, msg.angular.z))

def _on_mode(self, msg: String):
    mode = msg.data.strip().upper()
    if mode == 'AUTO':
        self._auto_mode = True
        self._write('AUTO')
    else:
        self._auto_mode = False
        self._write('OFF')

def stop_robot(self):
    if self.ser and self.ser.is_open:
        try:
            self.ser.write(b'OFF\n')
            self.ser.flush()
        except Exception:
            pass

def main(args=None):
    rclpy.init(args=args)
    node = UartBridge()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.stop_robot()
        if node.ser and node.ser.is_open:
            node.ser.close()
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()

```

- Điều khiển phần cứng: nano ~/ros2_ws/src/web_pose/launch/robot_bringup.launch.py

```

import os
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    # Construct the absolute path to the C++ filter config file
    filter_config = os.path.join(
        os.path.expanduser('~'), 'ros2_ws', 'config', 'laser_filter.yaml'
    )

    return LaunchDescription([
        # 1. LiDAR Driver Node
        Node(
            package='slidar_ros2',
            executable='slidar_node',
            name='slidar_node',
            parameters=[{
                'serial_port': '/dev/rplidar',
                'serial_baudrate': 115200,
                'frame_id': 'laser',
                'scan_mode': 'Standard'
            }],
            output='screen'
        ),

        # 2. C++ Laser Scan Filter Node (Cuts off noise < 15cm)
        Node(
            package='laser_filters',

```

```

        executable='scan_to_scan_filter_chain',
        name='laser_filter',
        parameters=[filter_config],
        remappings=[
            ('scan', '/scan'),
            ('scan_filtered', '/scan_filtered')
        ],
        output='screen'
    ),

    # 3. Static Transform: base_link -> laser (LiDAR height: 0.23m)
    Node(
        package='tf2_ros',
        executable='static_transform_publisher',
        arguments=[
            '--x', '0.0', '--y', '0.0', '--z', '0.23',
            '--yaw', '0', '--pitch', '0', '--roll', '0',
            '--frame-id', 'base_link', '--child-frame-id', 'laser'
        ],
        output='screen'
    ),

    # 4. Static Transform: base_link -> imu_link (IMU height: 0.09m)
    Node(
        package='tf2_ros',
        executable='static_transform_publisher',
        arguments=[
            '--x', '0.0', '--y', '0.0', '--z', '0.09',
            '--yaw', '0', '--pitch', '0', '--roll', '0',
            '--frame-id', 'base_link', '--child-frame-id', 'imu_link'
        ],
        output='screen'
    ),

    # 5. MPU6050 IMU Driver Node
    Node(
        package='web_pose',
        executable='mpu6050_driver',
        name='mpu6050_driver',
        output='screen'
    ),

    # 6. STM32 UART Bridge Node
    Node(
        package='web_pose',
        executable='uart_bridge',
        name='uart_bridge',
        output='screen'
    )
)
})

```

- Bổ sung config: nano ~/ros2_ws/src/web_pose/setup.py

```

from setuptools import find_packages, setup
import os
from glob import glob

package_name = 'web_pose'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'), glob('launch/*.py')),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='ubuntu',
    maintainer_email='ubuntu@todo.todo',
    description='Web pose package',
    license='Apache-2.0',
    extras_require={
        'test': [
            'pytest',
        ],
    },
    entry_points={
        'console_scripts': [
            'mpu6050_driver = web_pose.mpu6050_driver:main',

```

```
        'uart_bridge = web_pose.uart_bridge:main',  
    ],  
    },  
)
```

1.11. Build package:

```
cd ~/ros2_ws && colcon build --packages-select web_pose --symlink-install && source ~/.bashrc  
ros2 pkg list | grep web_pose
```

1.12. CycloneDDS Cross-Machine (Pi ↔ Ubuntu VM)

```
sudo apt update  
sudo apt install -y avahi-daemon  
sudo systemctl enable --now avahi-daemon
```

```
cat > ~/cyclone_dds.xml << 'EOF'  
<?xml version="1.0" encoding="utf-8"?>  
<CycloneDDS>  
  <Domain>  
    <General>  
      <NetworkInterfaceAddress>auto</NetworkInterfaceAddress>  
    </General>  
    <Discovery>  
      <Peers>  
        <Peer address="vmuser-virtual-machine.local"/>  
      </Peers>  
    </Discovery>  
  </Domain>  
</CycloneDDS>  
EOF
```

```
echo 'export CYCLONEDDS_URI=file:///home/ubuntu/cyclone_dds.xml' >> ~/.bashrc  
source ~/.bashrc
```

2.1. Cấu hình phần cứng camera IMX219:

```
sudo sed -i '/camera_auto_detect/d' /boot/firmware/config.txt  
sudo sed -i '/dtoverlay=imx219/d' /boot/firmware/config.txt  
echo "dtoverlay=imx219" | sudo tee -a /boot/firmware/config.txt  
sudo sed -i -e 's/ cma=[^ ]*/g' -e '1s/$/ cma=128M/' /boot/firmware/cmdline.txt
```

2.2. Udev DMA Heap:

```
echo 'SUBSYSTEM=="dma_heap", GROUP="video", MODE="0666" \  
| sudo tee /etc/udev/rules.d/99-dma-heap.rules  
  
sudo tee /etc/systemd/system/dma-heap-cma.service << 'EOF'  
[Unit]  
Description=Create dma_heap linux,cma symlink for rpical-apps  
After=local-fs.target  
[Service]  
Type=oneshot  
ExecStart=/bin/ln -sf /dev/dma_heap/reserved /dev/dma_heap/linux,cma  
RemainAfterExit=yes  
[Install]  
WantedBy=multi-user.target  
EOF  
  
sudo systemctl daemon-reload  
sudo systemctl enable dma-heap-cma.service
```


2.3. Build libcamera & rpicas-apps:

```
sudo apt update
sudo apt install -y \
  clang meson ninja-build pkg-config cmake \
  libyaml-dev python3-yaml python3-ply python3-jinja2 openssl \
  libdw-dev libunwind-dev libudev-dev \
  libgstreamer1.0-dev libgstreamer-plugins-base1.0-dev \
  libpython3-dev pybind11-dev libevent-dev libtiff-dev \
  libltng-ust-dev ltng-tools libgtest-dev \
  libboost-dev libboost-program-options-dev \
  libdrm-dev libexif-dev libjpeg-dev libpng-dev

sudo pip3 install --upgrade meson
```

```
# libcamera
cd ~
git clone https://github.com/raspberrypi/libcamera.git
cd libcamera
meson setup build --buildtype=release \
  -Dpipelines=rpi/vc4,rpi/pisp \
  -Dipas=rpi/vc4,rpi/pisp \
  -Dv4l2=true \
  -Dgstreamer=enabled \
  -Dtest=false \
  -Dlc-compliance=disabled \
  -Dcam=disabled \
  -Dqcam=disabled \
  -Ddocumentation=disabled \
  -Dpyscamera=enabled
ninja -C build -j2
sudo ninja -C build install
sudo ldconfig
```

```
# rpicas-apps
cd ~
git clone https://github.com/raspberrypi/rpicas-apps.git
cd rpicas-apps
meson setup build \
  -Denable_libav=disabled \
  -Denable_drm=disabled \
  -Denable_egl=disabled \
  -Denable_qt=disabled \
  -Denable_opencv=disabled \
  -Denable_tfite=disabled \
  -Denable_hailo=disabled
meson compile -C build -j2
sudo meson install -C build
sudo ldconfig

sudo usermod -aG video,render ubuntu
```

2.4. Tạo luồng stream camera:

mkdir -p ~/web

nano ~/web/cam_stream.py

```
#!/usr/bin/env python3
"""
MJPEG Streaming Server using rpicas-vid.
Runs independently of ROS 2, optimized for low latency and memory safety.
"""
import subprocess
import threading
import time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

WIDTH, HEIGHT, FPS = 640, 480, 30
BOUNDARY = b'jpgboundary'
PORT = 8080
MAX_BUF_SIZE = 1048576 # 1MB limit to prevent memory leak

class CamStream:
    def __init__(self):
        self.frame = b""
        self.condition = threading.Condition()
        self.proc = self._start_rpicas()
```

```

threading.Thread(target=self._read_frames, daemon=True).start()
print(f'[CAM] rpicas-vid started at {WIDTH}x{HEIGHT}@{FPS}fps')

def _start_rpicas(self):
    """Helper method to start or restart the rpicas-vid process."""
    return subprocess.Popen([
        'rpicas-vid', '-t', '0',
        '--width', str(WIDTH), '--height', str(HEIGHT),
        '--framerate', str(FPS),
        '--codec', 'mjpeg', '--nopreview', '-o', '-'
    ], stdout=subprocess.PIPE, stderr=subprocess.DEVNULL)

def _read_frames(self):
    buf = b''
    while True:
        chunk = self.proc.stdout.read(4096)
        if not chunk:
            self._restart()
            buf = b''
            continue

        buf += chunk

        # Memory leak protection: clear buffer if it grows too large
        if len(buf) > MAX_BUF_SIZE:
            print(f'[CAM] Warning: Buffer overflow detected. Flushing buffer.')
            buf = b''
            continue

        while True:
            start = buf.find(b'\xff\xd8')
            if start == -1:
                break
            end = buf.find(b'\xff\xd9', start + 2)
            if end == -1:
                break

            # Extract the complete JPEG frame
            with self.condition:
                self.frame = buf[start:end+2]
                self.condition.notify_all()

            # Truncate buffer to process next frame
            buf = buf[end+2:]

def _restart(self):
    print(f'[CAM] rpicas-vid crashed, restarting in 2s...')
    try:
        self.proc.kill()
    except Exception:
        pass
    time.sleep(2)
    self.proc = self._start_rpicas()
    print(f'[CAM] rpicas-vid restarted successfully')

def get_frame(self):
    with self.condition:
        # Wait up to 1 second for a new frame
        self.condition.wait(timeout=1.0)
    return self.frame

cam = CamStream()

class StreamHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path.split('?')[0] == '/stream.mjpg':
            self.send_response(200)
            self.send_header('Content-Type', f'multipart/x-mixed-replace; boundary={BOUNDARY.decode()}')
            self.send_header('Access-Control-Allow-Origin', '**')
            self.end_headers()
            try:
                while True:
                    frame = cam.get_frame()
                    if frame:
                        self.wfile.write(b'--' + BOUNDARY + b'\r\n')
                        self.wfile.write(b'Content-Type: image/jpeg\r\n\r\n')
                        self.wfile.write(frame)
                        self.wfile.write(b'\r\n')
            except Exception:
                # Client disconnected
                pass
        else:
            self.send_error(404)

def log_message(self, *args):

```

```
# Disable default logging to keep terminal output clean
pass

if __name__ == '__main__':
    print(f'[CAM] MJPEG server is running: http://0.0.0.0:{PORT}/stream.mjpg')
    try:
        ThreadingHTTPServer(('', PORT), StreamHandler).serve_forever()
    except KeyboardInterrupt:
        print("\n[CAM] Shutting down...")
        try:
            cam.proc.kill()
        except Exception:
            pass
```

3.1. Kết hợp pi và stm32 qua uart gpio:

```
sudo nano /boot/firmware/config.txt
```

```
dtoverlay=disable-bt
```

- Vô hiệu hóa service chiếm quyền UART:

```
sudo systemctl mask serial-getty@ttyAMA0.service
```

- Kiểm tra thêm cmdline.txt: nếu file /boot/firmware/cmdline.txt có chuỗi console=serial0,115200, kernel sẽ đổ log boot ra chính UART nối STM32 → STM32 có thể nhận nhiều lúc khởi động. Gỡ bỏ nó:

```
sudo sed -i 's/console=serial0,[0-9]* //' /boot/firmware/cmdline.txt
```

3.2. Mở cổng UFW

```
sudo ufw allow 8000 # web dashboard
sudo ufw allow 8080 # camera MJPEG
sudo ufw reload
```

- Rebuild:

```
cd ~/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --packages-select web_pose_slidar_ros2 --symlink-install
source ~/.bashrc
source ~/ros2_ws/install/setup.bash

sudo reboot
```

4. Web dashboard:

```
cd ~/web
curl -L -o roslib.min.js "https://cdn.jsdelivr.net/npm/roslib@1.3.0/build/roslib.min.js"
nano ~/web/index.html
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Robot An Ninh — SLAM Dashboard</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
```

```

<link href="https://fonts.googleapis.com/css2?family=Share+Tech+Mono&family=Exo+2:wght@300;400;600;700&display=swap"
rel="stylesheet">
<script src="/roslib.min.js"></script>
<style>
:root {
  --bg:#080c10; --surface:#0d1117; --panel:#111820; --border:#1e2d3d;
  --accent:#00d4ff; --accent2:#00ff88; --danger:#e94560;
  --text:#c9d1d9; --muted:#4a5568;
  --mono:'Share Tech Mono',monospace; --body:'Exo 2',sans-serif;
}
*{box-sizing:border-box;margin:0;padding:0;}
body{background:var(--bg);color:var(--text);font-family:var(--body);height:100vh;display:flex;flex-direction:column;overflow:hidden;}
body::before{content:"";position:fixed;inset:0;background:repeating-linear-gradient(0deg,transparent,transparent 2px,rgba(0,212,255,0.012) 2px,rgba(0,212,255,0.012) 4px);pointer-events:none;z-index:1000;}

/* — Header — */
header{background:var(--surface);padding:0 14px;height:44px;display:flex;align-items:center;gap:10px;border-bottom:1px solid var(--border);flex-shrink:0;position:relative;}

header::after{content:"";position:absolute;bottom:0;left:0;right:0;height:1px;background:linear-gradient(90deg,transparent,var(--accent),transparent);opacity:.4;}
.status-dot{width:8px;height:8px;border-radius:50%;background:var(--danger);flex-shrink:0;transition:background .4s;}
.status-dot.ok{background:var(--accent2);animation:pulse-green 2s infinite;}
@keyframes pulse-green{0%{box-shadow:0 0 0 6px rgba(0,255,136,.6)}70%{box-shadow:0 0 0 6px rgba(0,255,136,0)}100%{box-shadow:0 0 0 6px rgba(0,255,136,0)}}
.logo{font-family:var(--mono);font-size:13px;color:var(--accent);letter-spacing:2px;text-transform:uppercase;}
.logo span{color:var(--danger);}
#conn-status{font-family:var(--mono);font-size:10px;color:var(--muted);letter-spacing:1px;}
.header-right{margin-left:auto;display:flex;align-items:center;gap:12px;}
.header-time{font-family:var(--mono);font-size:11px;color:var(--accent);opacity:.7;}

/* — Main layout — */
main{flex:1;display:flex;overflow:hidden;}

/* — Full-screen camera area — */
.cam-area{flex:1;position:relative;background:#000;overflow:hidden;min-width:0;border-right:1px solid var(--border);}

.view-label{position:absolute;bottom:10px;right:12px;font-family:var(--mono);font-size:9px;color:var(--accent);letter-spacing:2px;text-transform:uppercase;opacity:.5;z-index:10;pointer-events:none;}
#camera-img{width:100%;height:100%;object-fit:cover;display:none;}
.camera-offline{position:absolute;inset:0;display:flex;flex-direction:column;align-items:center;justify-content:center;gap:8px;}
.camera-offline .icon{font-size:28px;opacity:.2;}
.camera-offline p{font-family:var(--mono);font-size:9px;letter-spacing:2px;text-transform:uppercase;color:var(--muted);opacity:.5;}

/* — Circular minimap (top-left overlay) — */
#minimap-wrap{
  position:absolute;top:14px;left:14px;
  width:150px;height:150px;border-radius:50%;
  overflow:hidden;cursor:pointer;z-index:20;
  border:2px solid rgba(0,212,255,.45);
  background:rgba(8,12,16,.9);
  box-shadow:0 0 24px rgba(0,212,255,.15), 0 4px 16px rgba(0,0,0,.6);
  transition:border-color .2s, box-shadow .2s;
}
#minimap-wrap:hover{
  border-color:var(--accent);
  box-shadow:0 0 32px rgba(0,212,255,.3), 0 4px 20px rgba(0,0,0,.7);
}
#miniCanvas{width:100%;height:100%;display:block;}
.mini-badge{
  position:absolute;bottom:0;left:0;right:0;text-align:center;
  padding:9px 0 7px;
  background:linear-gradient(to top,rgba(8,12,16,.9),transparent);
  font-family:var(--mono);font-size:7px;letter-spacing:2px;
  color:var(--accent);pointer-events:none;
}

/* — Full-screen map modal (covers camera area only) — */
#map-modal{
  display:none;position:absolute;inset:0;z-index:40;
  background:rgba(5,8,16,.96);flex-direction:column;
}
#map-modal.open{display:flex;}
#modal-hdr{
  height:44px;flex-shrink:0;display:flex;align-items:center;gap:10px;
  padding:0 14px;background:var(--surface);border-bottom:1px solid var(--border);
}
.modal-ttl{font-family:var(--mono);font-size:9px;color:var(--accent);letter-spacing:2px;text-transform:uppercase;}
#modal-close{
  margin-left:auto;
  background:rgba(233,69,96,.1);border:1px solid rgba(233,69,96,.4);
  border-radius:4px;color:var(--danger);font-family:var(--mono);
  font-size:9px;letter-spacing:1px;padding:6px 14px;cursor:pointer;
}
#modal-close:hover{background:rgba(233,69,96,.2);}

```

```

#modal-canvas{flex:1;display:block;width:100%;min-height:0;image-rendering:pixelated;}

/* — Sidebar — */
aside{width:210px;display:flex;flex-direction:column;background:var(--surface);overflow-y:auto;flex-shrink:0;}
aside::-webkit-scrollbar{width:3px;}
aside::-webkit-scrollbar-thumb{background:var(--border);}
.card{border-bottom:1px solid var(--border);padding:12px 14px;}
.card-title{font-family:var(--mono);font-size:9px;color:var(--muted);letter-spacing:2px;text-transform:uppercase;margin-bottom:8px;}
.metric-big{font-family:var(--mono);font-size:26px;color:var(--accent);line-height:1;}
.metric-unit{font-family:var(--mono);font-size:10px;color:var(--muted);margin-top:3px;}
.pose-grid{display:grid;grid-template-columns:1fr 1fr;gap:5px;}
.pose-item{background:var(--panel);border:1px solid var(--border);border-radius:4px;padding:5px 7px;}
.pose-key{font-family:var(--mono);font-size:9px;color:var(--accent);opacity:.6;}
.pose-num{font-family:var(--mono);font-size:12px;color:var(--text);}

/* Toggle switches */
.toggle-row{display:flex;align-items:center;gap:8px;padding:3px 0;}
.toggle-label{font-family:var(--mono);font-size:10px;color:var(--muted);}
.toggle-switch{position:relative;width:32px;height:18px;cursor:pointer;}
.toggle-switch input{opacity:0;width:0;height:0;}
.toggle-track{position:absolute;inset:0;background:var(--panel);border:1px solid var(--border);border-radius:9px;transition:background
2s,border-color .2s;}

.toggle-track::after{content:"";position:absolute;top:2px;left:2px;width:12px;height:12px;border-radius:50%;background:var(--muted);transition:tr
ansform .2s,background .2s;}
.toggle-switch input:checked+.toggle-track{background:rgba(0,212,255,.15);border-color:var(--accent);}
.toggle-switch input:checked+.toggle-track::after{transform:translateX(14px);background:var(--accent);}

/* Map save button */
.map-btn{background:var(--panel);border:1px solid
var(--border);border-radius:4px;color:var(--accent);font-family:var(--mono);font-size:9px;letter-spacing:1px;padding:5px
8px;cursor:pointer;width:100%;margin-top:4px;}
.map-btn.active{background:rgba(0,212,255,.15);}
.save-hint{font-family:var(--mono);font-size:8px;color:var(--muted);margin-top:5px;line-height:1.5;}

/* — D-pad cross layout — */
.controls-grid {
display: grid;
grid-template-columns: repeat(3, 1fr);
grid-template-areas:
" . fwd ."
"lft bck rgt"
"ap ap ap";
gap: 4px;
margin-top: 4px;
}
#btn-fwd { grid-area: fwd; }
#btn-left { grid-area: lft; }
#btn-back { grid-area: bck; }
#btn-right { grid-area: rgt; }
#btn-auto { grid-area: ap; }

.ctrl-btn {
background:var(--panel);border:1px solid var(--border);border-radius:4px;
color:var(--text);font-family:var(--mono);font-size:14px;
padding:8px 4px;cursor:pointer;text-align:center;
transition:background .12s,border-color .12s,color .12s;user-select:none;
}
.ctrl-btn.active, .ctrl-btn.pressed {
background:var(--accent);border-color:var(--accent);color:var(--bg);
}
.auto-btn {
background:rgba(0,255,136,.1);border-color:var(--accent2);
color:var(--accent2);font-size:10px;letter-spacing:1px;padding:8px;
}
.auto-btn.active {
background:rgba(233,69,96,.15);border-color:var(--danger);color:var(--danger);
}
.auto-btn.active { opacity:.8; }

/* Joystick */
.joystick-wrap{display:flex;flex-direction:column;align-items:center;gap:6px;padding-top:4px;}
#joystick-canvas{display:block;cursor:grab;touch-action:none;border-radius:50%;}
.joystick-vals{font-family:var(--mono);font-size:9px;color:var(--muted);letter-spacing:1px;}
</style>
</head>
<body>

<header>
<div class="status-dot" id="dot"></div>
<div class="logo">ROBOT <span>AN NINH</span></div>
<span id="conn-status">CONNECTING...</span>
<div class="header-right">
<span class="header-time" id="clock">--:--:--</span>
</div>

```

```

</header>

<main>
  <!-- Full-screen camera -->
  <div class="cam-area">
    <span class="view-label">> CAMERA · LIVE</span>
    <div class="camera-offline" id="cam-offline">
      <div class="icon"><img alt="Offline camera icon" data-bbox="245 161 260 171"/></div><p>camera offline</p>
    </div>
    <img id="camera-img" alt="camera">

    <!-- Circular minimap — click to expand -->
    <div id="minimap-wrap" title="Click to expand map">
      <canvas id="miniCanvas" width="150" height="150"></canvas>
      <div class="mini-badge">> MAP <img alt="Map icon" data-bbox="345 231 355 241"/></div>
    </div>

    <!-- Full-screen map modal (covers camera area only) -->
    <div id="map-modal">
      <div id="modal-hdr">
        <span class="modal-ttl">> SLAM MAP · /map</span>
        <button id="modal-close">X Collapse</button>
      </div>
      <canvas id="modal-canvas"></canvas>
    </div>
  </div>

  <aside>
    <!-- Speed / Angular rate -->
    <div style="display:flex;border-bottom:1px solid var(--border);">
      <div style="flex:1;padding:12px 10px;border-right:1px solid var(--border);">
        <div class="card-title">Speed</div>
        <div class="metric-big" id="speed" style="font-size:20px;">0.00</div>
        <div class="metric-unit">m/s</div>
      </div>
      <div style="flex:1;padding:12px 10px;">
        <div class="card-title">Angular</div>
        <div class="metric-big" id="angspeed" style="font-size:20px;">0.00</div>
        <div class="metric-unit">rad/s</div>
      </div>
    </div>

    <!-- Robot pose -->
    <div class="card">
      <div class="card-title">Position · Heading</div>
      <div class="pose-grid">
        <div class="pose-item"><div class="pose-key">X</div><div class="pose-num" id="px">—</div></div>
        <div class="pose-item"><div class="pose-key">Y</div><div class="pose-num" id="py">—</div></div>
        <div class="pose-item" style="grid-column:1/-1">
          <div class="pose-key">YAW (°)</div>
          <div class="pose-num" id="pyaw">—°</div>
        </div>
      </div>
    </div>

    <!-- Display toggles -->
    <div class="card">
      <div class="card-title">Display</div>
      <div class="toggle-row">
        <label class="toggle-switch"><input type="checkbox" id="scan-chk" checked><div class="toggle-track"></div></label>
        <span class="toggle-label" id="scan-lbl">Scan ON</span>
      </div>
      <div class="toggle-row">
        <label class="toggle-switch"><input type="checkbox" id="grid-chk" checked><div class="toggle-track"></div></label>
        <span class="toggle-label" id="grid-lbl">Grid 1m ON</span>
      </div>
    </div>

    <!-- Save map -->
    <div class="card">
      <div class="card-title">Save Map</div>
      <button class="map-btn" id="btn-save"><img alt="Save icon" data-bbox="358 771 373 781"/> Serialize map</button>
      <div class="save-hint" id="save-status">Flush pose graph → SD card</div>
    </div>

    <!-- D-pad controls -->
    <div class="card">
      <div class="card-title">Controls</div>
      <div class="controls-grid">
        <button class="ctrl-btn" id="btn-fwd" data-lx="0.2" data-az="0" >↑</button>
        <button class="ctrl-btn" id="btn-left" data-lx="0" data-az="0.5" >←</button>
        <button class="ctrl-btn" id="btn-back" data-lx="-0.2" data-az="0" >↓</button>
        <button class="ctrl-btn" id="btn-right" data-lx="0" data-az="-0.5" >→</button>
        <button class="ctrl-btn auto-btn" id="btn-auto">> AUTO NAV</button>
      </div>
    </div>
  </aside>

```

```

</div>

<!-- Joystick — flex:1 fills remaining sidebar space -->
<div class="card" style="flex:1;">
  <div class="card-title">Joystick</div>
  <div class="joystick-wrap">
    <canvas id="joystick-canvas" width="140" height="140"></canvas>
    <div class="joystick-vals" id="joy-vals">lx: 0.00 · az: 0.00</div>
  </div>
</div>
</aside>
</main>

<script>
// — Configuration —————
const PI_HOST    = location.hostname;
const MAC_HOST   = 'vmuser-virtual-machine.local';
const ROS_URL    = `ws://${MAC_HOST}:9090`;
const CAM_URL    = `http://${PI_HOST}:8080/stream.mjpg`;
const RECONNECT_MS = 3000;
const BASE_PPM   = 40;
const MINI_PPM   = 12;
const LIDAR_X_OFFSET = 0.0;
const MAX_LINEAR  = 0.3;
const MAX_ANGULAR = 0.8;

// — Clock —————
setInterval(() => {
  document.getElementById('clock').textContent = new Date().toLocaleTimeString('en-GB', {hour12:false});
}, 1000);

// — ROS Connection State —————
const dot    = document.getElementById('dot');
const connStat = document.getElementById('conn-status');
let _cv      = null;
let _mode    = null;
let autoMode = false;
let _rosOK   = false;
let topics   = {};

const ros = new ROSLIB.Ros({ url: ROS_URL });

ros.on('connection', () => {
  _rosOK = true;
  dot.classList.add('ok');
  connStat.textContent = `CONNECTED · ${MAC_HOST}:9090`;
  subscribeTopics();
});

ros.on('error', () => {
  _rosOK = false;
  connStat.textContent = `ERROR · ${MAC_HOST}:9090`;
});

ros.on('close', () => {
  _rosOK = false;
  dot.classList.remove('ok');
  connStat.textContent = `DISCONNECTED · retry ${RECONNECT_MS/1000}s...`;

  if (autoMode) {
    autoMode = false;
    const b = document.getElementById('btn-auto');
    b.classList.remove('active');
    b.textContent = '▶ AUTO NAV';
  }

  unsubscribeTopics();
  setTimeout(() => {
    connStat.textContent = `RECONNECTING → ${MAC_HOST}:9090`;
    ros.connect(ROS_URL);
  }, RECONNECT_MS);
});

function unsubscribeTopics() {
  for (const key in topics) {
    if (topics[key]) topics[key].unsubscribe();
  }
  topics = {};
}

// — Publishers —————
function sendVel(lx, az) {
  if (!_rosOK) return;
  if (!_cv) _cv = new ROSLIB.Topic({ ros, name:'/cmd_vel', messageType:'geometry_msgs/Twist' });
  _cv.publish(new ROSLIB.Message({ linear:{x:lx, y:0, z:0}, angular:{x:0, y:0, z:az} }));
}

```

```

}

function sendMode(mode) {
  if (!_rosOK) return;
  if (!_mode) _mode = new ROSLIB.Topic({ ros, name: '/robot_mode', messageType: 'std_msgs/String' });
  _mode.publish(new ROSLIB.Message({ data: mode }));
}

// — SLAM Map State —
let mapInfo=null, mapBitmap=null, robotPose=null;
let scanPoints=[], showScan=true, showGrid=true, _raf=false;
let mapModalOpen = false;

const miniCanvas = document.getElementById('miniCanvas');
const miniCtx = miniCanvas.getContext('2d');
const modalCanvas = document.getElementById('modal-canvas');
const modalCtx = modalCanvas.getContext('2d');

function scheduleRedraw() {
  if (_raf) return; _raf=true;
  requestAnimationFrame(() => {
    _raf=false;
    redrawMini();
    if (mapModalOpen) redrawModal();
  });
}

function buildBitmap(msg) {
  const w = msg.info.width, h = msg.info.height;
  const img = new ImageData(w, h), d = msg.data;
  for (let row = 0; row < h; row++) {
    const ro = (h - 1 - row) * w, co = row * w * 4;
    for (let col = 0; col < w; col++) {
      const v = d[ro + col]; let r, g, b;
      if (v === -1) { r = 30; g = 40; b = 50; }
      else if (v === 0) { r = 230; g = 235; b = 240; }
      else { r = 10; g = 20; b = 30; }
      const p = co + col * 4;
      img.data[p] = r; img.data[p+1] = g; img.data[p+2] = b; img.data[p+3] = 255;
    }
  }
  return createImageBitmap(img);
}

function renderMap(ctx, W, H, ppm) {
  const wcx = robotPose ? robotPose.x : (mapInfo ? mapInfo.origin.position.x + mapInfo.width * mapInfo.resolution / 2 : 0);
  const wcy = robotPose ? robotPose.y : (mapInfo ? mapInfo.origin.position.y + mapInfo.height * mapInfo.resolution / 2 : 0);

  ctx.fillStyle = '#050810'; ctx.fillRect(0,0,W,H);

  if (showGrid) {
    const G = 1.0;
    const wL = wcx - W/(2*ppm), wR = wcx + W/(2*ppm), wT = wcy + H/(2*ppm), wB = wcy - H/(2*ppm);
    ctx.save(); ctx.setLineDash([2,4]); ctx.lineWidth = 0.5;
    for (let wx = Math.ceil(wL/G)*G; wx <= wR+G; wx += G) {
      const cx = W/2 + (wx - wcx)*ppm;
      ctx.strokeStyle = Math.round(wx)%5===0 ? 'rgba(0,212,255,0.18)' : 'rgba(0,212,255,0.07)';
      ctx.beginPath(); ctx.moveTo(cx,0); ctx.lineTo(cx,H); ctx.stroke();
      if (ppm >= 20 && Math.round(wx)%5===0 && Math.round(wy)!==0) {
        ctx.fillStyle = 'rgba(0,212,255,0.4)'; ctx.font = "9px 'Share Tech Mono',monospace";
        ctx.fillText(Math.round(wx)+'m', cx+2, H-4);
      }
    }
    for (let wy = Math.floor(wT/G)*G; wy >= wB-G; wy -= G) {
      const cy = H/2 - (wy - wcy)*ppm;
      ctx.strokeStyle = Math.round(wy)%5===0 ? 'rgba(0,212,255,0.18)' : 'rgba(0,212,255,0.07)';
      ctx.beginPath(); ctx.moveTo(0,cy); ctx.lineTo(W,cy); ctx.stroke();
      if (ppm >= 20 && Math.round(wy)%5===0 && Math.round(wy)!==0) {
        ctx.fillStyle = 'rgba(0,212,255,0.4)'; ctx.font = "9px 'Share Tech Mono',monospace";
        ctx.fillText(Math.round(wy)+'m', 2, cy-2);
      }
    }
  }
  ctx.setLineDash([]); ctx.restore();
}

if (mapBitmap && mapInfo) {
  const res = mapInfo.resolution, mw = mapInfo.width, mh = mapInfo.height;
  const ox = mapInfo.origin.position.x, oy = mapInfo.origin.position.y;
  ctx.imageSmoothingEnabled = false;
  ctx.drawImage(mapBitmap, W/2 + (ox - wcx)*ppm, H/2 - (oy - wcy)*ppm - mh*res*ppm, mw*res*ppm, mh*res*ppm);
}

if (showScan && scanPoints.length > 0) {
  const d = ppm >= 20 ? 2.5 : 1.5;
  ctx.save(); ctx.fillStyle = '#00ffff';

```



```

    for (const pt of scanPoints) ctx.fillRect(W/2 + (pt.x - wcx)*ppm - d/2, H/2 - (pt.y - wcy)*ppm - d/2, d, d);
    ctx.restore();
  }

  if (robotPose) {
    const cx = W/2, cy = H/2, r = ppm >= 20 ? 8 : 5, yaw = robotPose.yaw;
    ctx.save();
    if (ppm >= 20) {
      ctx.beginPath(); ctx.arc(cx,cy,r+4,0,2*Math.PI);
      ctx.strokeStyle = 'rgba(233,69,96,0.3)'; ctx.lineWidth = 2; ctx.stroke();
    }
    ctx.beginPath(); ctx.arc(cx,cy,r,0,2*Math.PI);
    ctx.fillStyle = '#e94560'; ctx.fill();
    ctx.strokeStyle = 'rgba(255,255,255,0.7)'; ctx.lineWidth = 1.5; ctx.stroke();
    ctx.beginPath(); ctx.moveTo(cx,cy);
    ctx.lineTo(cx + r*2.8*Math.cos(yaw), cy - r*2.8*Math.sin(yaw));
    ctx.strokeStyle = '#fff'; ctx.lineWidth = 2; ctx.lineCap = 'round'; ctx.stroke();
    ctx.restore();
  }

  if (ppm >= 20) {
    const sx = W - ppm - 16, sy = H - 16;
    ctx.save(); ctx.strokeStyle = '#00d4ff'; ctx.lineWidth = 2; ctx.lineCap = 'square';
    ctx.beginPath(); ctx.moveTo(sx,sy); ctx.lineTo(sx+ppm,sy); ctx.stroke();
    ctx.beginPath(); ctx.moveTo(sx,sy-4); ctx.lineTo(sx,sy+4); ctx.stroke();
    ctx.beginPath(); ctx.moveTo(sx+ppm,sy-4); ctx.lineTo(sx+ppm,sy+4); ctx.stroke();
    ctx.fillStyle = '#00d4ff'; ctx.font = "9px 'Share Tech Mono',monospace";
    ctx.textAlign = 'center'; ctx.fillText('1 m', sx+ppm/2, sy-6);
    ctx.textAlign = 'left'; ctx.restore();
  }
}

function redrawMini() {
  miniCanvas.width = miniCanvas.offsetWidth || 150;
  miniCanvas.height = miniCanvas.offsetHeight || 150;
  renderMap(miniCtx, miniCanvas.width, miniCanvas.height, MINI_PPM);
}

function redrawModal() {
  const W = modalCanvas.offsetWidth, H = modalCanvas.offsetHeight;
  if (!W || !H) return;
  modalCanvas.width = W; modalCanvas.height = H;
  renderMap(modalCtx, W, H, BASE_PPM);
}

// — ROS Subscriptions —
function subscribeTopics() {
  topics.map = new ROSLIB.Topic({ ros, name:'/map', messageType:'nav_msgs/OccupancyGrid' });
  topics.map.subscribe(async msg => {
    mapInfo = msg.info; mapBitmap = await buildBitmap(msg); scheduleRedraw();
  });

  topics.scan = new ROSLIB.Topic({ ros, name:'/scan_filtered', messageType:'sensor_msgs/LaserScan', throttle_rate:100, queue_length:1 });
  topics.scan.subscribe(msg => {
    const pts=[], am=msg.angle_min, ai=msg.angle_increment, rm=msg.range_min, rx=msg.range_max;
    const px=robotPose?robotPose.x:0, py=robotPose?robotPose.y:0, yaw=robotPose?robotPose.yaw:0;
    const c=Math.cos(yaw), s=Math.sin(yaw);
    for (let i=0; i<msg.ranges.length; i++) {
      const r = msg.ranges[i];
      if (!isFinite(r) || r<rm || r>rx) continue;
      const a = am + i*ai, bx = r*Math.cos(a) + LIDAR_X_OFFSET, by = r*Math.sin(a);
      pts.push({x: px + c*bx - s*by, y: py + s*bx + c*by});
    }
    scanPoints = pts; scheduleRedraw();
  });

  topics.pose = new ROSLIB.Topic({ ros, name:'/robot_pose', messageType:'geometry_msgs/PoseStamped' });
  topics.pose.subscribe(msg => {
    const q = msg.pose.orientation;
    const yaw = Math.atan2(2*(q.w*q.z + q.x*q.y), 1 - 2*(q.y*q.y + q.z*q.z));
    robotPose = {x: msg.pose.position.x, y: msg.pose.position.y, yaw};
    document.getElementById('px').textContent = robotPose.x.toFixed(3);
    document.getElementById('py').textContent = robotPose.y.toFixed(3);
    document.getElementById('pyaw').textContent = (yaw*180/Math.PI).toFixed(1)+'°';
    if (mapInfo && mapBitmap) scheduleRedraw();
  });

  topics.odom = new ROSLIB.Topic({ ros, name:'/odom', messageType:'nav_msgs/Odometry' });
  topics.odom.subscribe(msg => {
    document.getElementById('speed').textContent = msg.twist.twist.linear.x.toFixed(2);
    document.getElementById('angspeed').textContent = msg.twist.twist.angular.z.toFixed(2);
  });
}

// — UI Events —
document.getElementById('minimap-wrap').addEventListener('click', () => {

```

```

mapModalOpen = true;
document.getElementById('map-modal').classList.add('open');
setTimeout(redrawModal, 10);
});
document.getElementById('modal-close').addEventListener('click', () => {
  mapModalOpen = false;
  document.getElementById('map-modal').classList.remove('open');
});
new ResizeObserver(() => { if (mapModalOpen) redrawModal(); }).observe(document.getElementById('modal-canvas'));

document.getElementById('scan-chk').addEventListener('change', e => {
  showScan = e.target.checked;
  document.getElementById('scan-lbl').textContent = showScan ? 'Scan ON' : 'Scan OFF';
  scheduleRedraw();
});
document.getElementById('grid-chk').addEventListener('change', e => {
  showGrid = e.target.checked;
  document.getElementById('grid-lbl').textContent = showGrid ? 'Grid 1m ON' : 'Grid OFF';
  scheduleRedraw();
});

document.getElementById('btn-save').addEventListener('click', () => {
  if (!_rosOK) { document.getElementById('save-status').textContent = '✗ Not connected'; return; }
  const fname = '/home/ubuntu/map_' + Date.now();
  document.getElementById('save-status').textContent = '💾 Saving...';
  new ROSLIB.Service({ros, name: '/slam_toolbox/serialize_pose_graph', serviceType: 'slam_toolbox/SerializePoseGraph'})
    .callService(new ROSLIB.ServiceRequest({filename: fname}),
      () => { document.getElementById('save-status').textContent = '✅' + fname; },
      err => { document.getElementById('save-status').textContent = '✗' + err; }
    );
});

const camImg = document.getElementById('camera-img');
const camOff = document.getElementById('cam-offline');
function loadCam() { camImg.src = CAM_URL + '?t=' + Date.now(); }
camImg.onload = () => { camImg.style.display = 'block'; camOff.style.display = 'none'; };
camImg.onerror = () => { camImg.style.display = 'none'; camOff.style.display = 'flex'; setTimeout(loadCam, 4000); };
loadCam();

document.querySelectorAll('.ctrl-btn[data-lx]').forEach(btn => {
  const lx = parseFloat(btn.dataset.lx), az = parseFloat(btn.dataset.az);
  const release = () => { btn.classList.remove('pressed'); sendVel(0,0); };
  btn.addEventListener('pointerdown', e => { e.preventDefault(); btn.classList.add('pressed'); sendVel(lx,az); });
  btn.addEventListener('pointerup', release);
  btn.addEventListener('pointerleave', () => { if (btn.classList.contains('pressed')) release(); });
  btn.addEventListener('pointercancel', release);
});

document.getElementById('btn-auto').addEventListener('click', () => {
  autoMode = !autoMode;
  const btn = document.getElementById('btn-auto');
  if (autoMode) {
    btn.classList.add('active');
    btn.textContent = '■ STOP AUTO';
    sendMode('AUTO');
  } else {
    btn.classList.remove('active');
    btn.textContent = '▶ AUTO NAV';
    sendMode('MANUAL');
    sendVel(0,0);
  }
});

// — Keyboard Controls —
const keys = new Set();
let isJoyActive = false; // Prevent Joystick & Keyboard conflicts

function clearDpad() {
  document.querySelectorAll('.ctrl-btn[data-lx]').forEach(b => b.classList.remove('pressed'));
}

document.addEventListener('keydown', e => {
  const k = e.key.toLowerCase(); // Fix CapsLock sticky bug
  if ([' ', 'arrowup', 'arrowdown', 'arrowleft', 'arrowright'].includes(k)) e.preventDefault();
  if (k === ' ') { keys.clear(); sendVel(0,0); clearDpad(); return; }
  if (isJoyActive || keys.has(k)) return;
  keys.add(k);
  applyKeys();
});

document.addEventListener('keyup', e => {
  const k = e.key.toLowerCase();
  keys.delete(k);
  if (!isJoyActive) applyKeys();
});

```

```

function applyKeys() {
  if (isJoyActive) return;

  const up = keys.has('arrowup') || keys.has('w');
  const down = keys.has('arrowdown') || keys.has('s');
  const left = keys.has('arrowleft') || keys.has('a');
  const right = keys.has('arrowright') || keys.has('d');

  let lx = 0, az = 0;
  if (up) lx = 0.2;
  if (down) lx = -0.2;
  if (left) az = 0.5;
  if (right) az = -0.5;
  sendVel(lx, az);

  clearDpad();
  if (up) document.getElementById('btn-fwd')?.classList.add('pressed');
  if (down) document.getElementById('btn-back')?.classList.add('pressed');
  if (left) document.getElementById('btn-left')?.classList.add('pressed');
  if (right) document.getElementById('btn-right')?.classList.add('pressed');
}

// — Analog Joystick —————
(function(){
  const jc = document.getElementById('joystick-canvas');
  const jx = jc.getContext('2d');
  const CX = 70, CY = 70, OR = 64, IR = 22, DEAD = 0.08;
  let active = false, sx = 0, sy = 0, af = null;

  function draw(){
    jx.clearRect(0,0,140,140);
    jx.beginPath(); jx.arc(CX,CY,OR,0,2*Math.PI);
    jx.strokeStyle = 'rgba(0,212,255,0.2)'; jx.lineWidth = 1.5; jx.stroke();
    jx.strokeStyle = 'rgba(0,212,255,0.07)'; jx.lineWidth = 1;
    jx.beginPath(); jx.moveTo(CX,CY-OR); jx.lineTo(CX,CY+OR); jx.stroke();
    jx.beginPath(); jx.moveTo(CX-OR,CY); jx.lineTo(CX+OR,CY); jx.stroke();
    jx.beginPath(); jx.arc(CX,CY,OR*.5,0,2*Math.PI);
    jx.strokeStyle = 'rgba(0,212,255,0.06)'; jx.stroke();

    const kx = CX + sx * OR, ky = CY + sy * OR;
    jx.beginPath(); jx.moveTo(CX,CY); jx.lineTo(kx,ky);
    jx.strokeStyle = active ? 'rgba(233,69,96,0.5)' : 'rgba(0,212,255,0.25)';
    jx.lineWidth = 1.5; jx.stroke();

    const g = jx.createRadialGradient(kx,ky,0,kx,ky,IR*1.6);
    g.addColorStop(0, active ? 'rgba(233,69,96,0.3)' : 'rgba(0,212,255,0.15)');
    g.addColorStop(1, 'transparent');
    jx.beginPath(); jx.arc(kx,ky,IR*1.6,0,2*Math.PI); jx.fillStyle = g; jx.fill();

    const col = active ? '#e94560' : '#00d4ff';
    jx.beginPath(); jx.arc(kx,ky,IR,0,2*Math.PI);
    jx.fillStyle = col + '33'; jx.fill();
    jx.strokeStyle = col; jx.lineWidth = 1.5; jx.stroke();

    jx.beginPath(); jx.arc(CX,CY,3,0,2*Math.PI);
    jx.fillStyle = 'rgba(0,212,255,0.3)'; jx.fill();
  }

  function pos(e){
    const rc = jc.getBoundingClientRect(), pt = e.touches ? e.touches[0] : e;
    return {x: (pt.clientX - rc.left - CX) / OR, y: (pt.clientY - rc.top - CY) / OR};
  }

  function clamp(x,y){
    const d = Math.sqrt(x*x + y*y);
    return d > 1 ? {x: x/d, y: y/d} : {x,y};
  }

  function applyJoy(){
    const lx = Math.abs(sy) > DEAD ? -sy * MAX_LINEAR : 0;
    const az = Math.abs(sx) > DEAD ? -sx * MAX_ANGULAR : 0;
    document.getElementById('joy-vals').textContent = `lx:${lx.toFixed(2)} · az:${az.toFixed(2)} `;
    sendVel(lx, az);
  }

  function start(e){
    e.preventDefault();
    active = true;
    isJoyActive = true; // Disable keyboard while joystick is active
    jc.style.cursor = 'grabbing';
    move(e);
  }

  function move(e){

```

```

    if(!active) return;
    e.preventDefault();
    const p = pos(e), c = clamp(p.x, p.y);
    sx = c.x; sy = c.y;
    cancelAnimationFrame(af);
    af = requestAnimationFrame(() => { draw(); applyJoy(); });
  }

  function end(){
    active = false;
    isJoyActive = false; // Re-enable keyboard after joystick released
    jc.style.cursor = 'grab';
    function sp(){
      sx *= .7; sy *= .7;
      if(Math.abs(sx) < .01 && Math.abs(sy) < .01){
        sx = sy = 0; draw(); applyJoy(); return;
      }
      draw(); applyJoy(); af = requestAnimationFrame(sp);
    }
    cancelAnimationFrame(af); af = requestAnimationFrame(sp);
  }

  jc.addEventListener('mousedown', start);
  window.addEventListener('mousemove', e => { if(active) move(e); });
  window.addEventListener('mouseup', end);
  jc.addEventListener('touchstart', start, {passive:false});
  jc.addEventListener('touchmove', move, {passive:false});
  jc.addEventListener('touchend', end);
  jc.addEventListener('touchcancel', end);

  draw();
  redrawMini();
})();
</script>
</body>
</html>

```

Cài trên Mac

1.1. Cài đặt cơ bản ubuntu:

- Cập nhật hệ điều hành:

```
sudo apt update && sudo apt upgrade -y  
sudo reboot
```

- Cài bộ đồng bộ thời gian:

```
sudo apt update && sudo apt install chrony -y
```

1.2. CycloneDDS Cross-Machine (Pi ↔ Mac VM)”

```
sudo apt update  
sudo apt install -y avahi-daemon  
sudo systemctl enable --now avahi-daemon
```

- Cấu hình mạng lỗi

```
cat > ~/cyclone_dds.xml << 'EOF'  
<?xml version="1.0" encoding="utf-8"?>  
<CycloneDDS>  
  <Domain>  
    <General>  
      <NetworkInterfaceAddress>auto</NetworkInterfaceAddress>  
    </General>  
    <Discovery>  
      <Peers>  
        <Peer address="robotanninh.local"/>  
      </Peers>  
    </Discovery>  
  </Domain>  
</CycloneDDS>  
EOF
```

1.3. Cài đặt ROS2:

- Thiết lập Locale

```
sudo apt update && sudo apt install locales -y  
sudo locale-gen en_US en_US.UTF-8  
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8  
export LANG=en_US.UTF-8
```

- Bật Universe:

```
sudo apt install software-properties-common -y  
sudo add-apt-repository universe -y
```

- Thêm key bảo mật và nguồn tải ROS 2:

```
sudo apt update && sudo apt install curl -y  
export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest | grep -F  
"tag_name" | awk -F"" '{print $4}')  
curl -L -o /tmp/ros2-apt-source.deb  
"https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SO  
URCE_VERSION}.${. /etc/os-release && echo ${UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"  
sudo dpkg -i /tmp/ros2-apt-source.deb
```

- Cài đặt ROS 2 Humble Desktop và các Tool:

```
sudo apt update && sudo apt upgrade -y  
sudo apt install ros-humble-desktop ros-dev-tools -y
```

1.4. Cài đặt các gói thuật toán SLAM, EKF và Nav2

```
sudo apt install ros-humble-slam-toolbox ros-humble-navigation2 ros-humble-nav2-bringup ros-humble-robot-localization  
ros-humble-rmw-cyclonedds-cpp ros-humble-rosbridge-suite -y
```

1.5. Cấu hình môi trường tự động:

```
echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc  
echo 'export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp' >> ~/.bashrc  
echo 'export ROS_DOMAIN_ID=42' >> ~/.bashrc  
source ~/.bashrc
```

1.6. Khởi tạo Workspace và tải mã nguồn thuật toán

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws/src  
  
git clone -b humble-devel https://github.com/Adlink-ROS/rf2o_laser_odometry.git  
  
ros2 pkg create mac_brain --build-type ament_python  
mkdir -p ~/ros2_ws/config  
mkdir -p ~/ros2_ws/src/mac_brain/launch
```